03. Exploratory Data Analysis

Inspect the metadata and text before creating similarity vectors.

Project: Movie Recommender System

Source archive supplied in this conversation: Movie-recommender-system-main.zip | Notebook: notebook.ipynb | App: app.py

EDA in the notebook

- Reads the two CSV files and inspects sample rows and shapes.
- Checks the movie and credits schemas through head() and dataframe info outputs.
- Selects seven fields and drops rows with null values.
- Parses JSON-like genres, keywords, cast and crew fields into Python lists.
- Samples transformed movie records and inspects the resulting tags for examples such as Avatar and Gandhi.

What EDA should establish

- Missingness by selected field and retained record count after dropna.
- Duplicate titles and duplicate movie IDs; join cardinality after merging.
- Distribution of genres and languages; frequency of keywords, actors and directors.
- Overview length and empty/very short descriptions.
- How many records have fewer than three cast members or no director.
- Common high-frequency tokens and rare tokens that may not be useful.

Important text-specific issues

A bag-of-words model can be dominated by frequent terms. CountVectorizer removes English stop words, but does not perform stemming or lemmatization. The notebook removes spaces inside names such as multi-word actors/directors (for example, joining the words together) to keep them as one token; this avoids splitting a name but creates a token vocabulary that may not match ordinary word forms.

EDA improvements

- Add explicit duplicate and null diagnostics before/after merge and parsing.
- Review outlier descriptions and malformed JSON records with try/except and logging.
- Compare token frequencies and document frequencies; identify generic terms shared by many films.
- Inspect similarity distributions and top neighbours for several representative films, not just one example.
- Check whether recommendations are dominated by franchise sequels or shared cast members.